

ASSIGNMENT-II

Student Name: PRIYANSHU

Branch: AIT-CSE-AIML

Semester: 5

Subject Name: Computer Networks

UID: 24BAI70700

Section/Group: 24AIT_KRG1_B

Subject Code: 24CST-339

Q1. What is an IP Address? Why do we need classes in case of IPv4 address? Explain each class, mention how many hosts and networks are possible in each class. Also explain subnetting and supernetting with examples.

Answer:

An **IP Address (Internet Protocol Address)** is a unique logical address assigned to a device connected to a computer network. It is used to identify a device and to provide the source and destination information required for communication over a network.

An **IPv4 address** is a 32-bit address divided into four 8-bit parts called octets. It is generally written in decimal form separated by dots.

Example:

192.168.1.10

IPv4 addresses were traditionally divided into different classes according to the size of the network and the number of hosts required. This classful addressing system helped organizations choose an appropriate address range for their networks.

IPv4 Address Classes

There are five classes of IPv4 addresses: **Class A, Class B, Class C, Class D and Class E.**

1. Class A

Class A addresses are designed for very large networks. The first octet is used for the network portion and the remaining three octets are used for hosts.

- Range: **1 to 126**
- Default subnet mask: **255.0.0.0**
- Network bits: **8**
- Host bits: **24**

ASSIGNMENT-II

- Number of networks: **126**
- Usable hosts per network:

$$2^{24} - 2 = 16,777,214$$

Example: 10.20.30.40

2. Class B

Class B addresses are designed for medium-sized networks. The first two octets represent the network and the remaining two represent the host.

- Range: **128 to 191**
- Default subnet mask: **255.255.0.0**
- Network bits: **16**
- Host bits: **16**
- Number of networks: **16,384**
- Usable hosts per network:

$$2^{16} - 2 = 65,534$$

Example: 172.16.10.5

3. Class C

Class C addresses are used for relatively small networks. The first three octets represent the network and the last octet represents the host.

- Range: **192 to 223**
- Default subnet mask: **255.255.255.0**
- Network bits: **24**
- Host bits: **8**
- Number of networks: **2,097,152**
- Usable hosts per network:

$$2^8 - 2 = 254$$

Example: 192.168.1.10

4. Class D

ASSIGNMENT-II

Class D addresses are used for **multicast communication**. They are not divided into network and host portions in the same way as Classes A, B and C.

- Range: **224 to 239**
- Used for multicast communication.

Example: 224.0.0.1

5. Class E

Class E addresses are reserved for **experimental and research purposes**.

- Range: **240 to 255**
- Not normally used for assigning addresses to regular hosts.

Summary

	Class Range	Default Mask	Hosts per Network
A	1–126	255.0.0.0	16,777,214
B	128–191	255.255.0.0	65,534
C	192–223	255.255.255.0	254
D	224–239	Multicast	—
E	240–255	Experimental	—

Subnetting

Subnetting is the process of dividing a large network into smaller logical networks called **subnets**. It is done by borrowing some bits from the host portion of an IP address and using them as network/subnet bits.

For example, consider the network:

192.168.1.0/24

If 2 bits are borrowed from the host portion, the new prefix becomes /26.

Number of subnets:

$$2^2 = 4$$

Remaining host bits:

ASSIGNMENT-II

$$8 - 2 = 6$$

Usable hosts per subnet:

$$2^6 - 2 = 62$$

Thus, the original network is divided into four smaller networks:

- 192.168.1.0/26
- 192.168.1.64/26
- 192.168.1.128/26
- 192.168.1.192/26

Subnetting helps in **efficient IP address utilization, reducing broadcast traffic and improving network management.**

Supernetting

Supernetting is the process of combining multiple smaller networks into one larger logical network. It is also called **route aggregation**.

For example, the following four networks:

192.168.0.0/24
192.168.1.0/24
192.168.2.0/24
192.168.3.0/24

can be combined into:

192.168.0.0/22

Supernetting reduces the number of routing table entries and makes routing more efficient.

Q2. What is the difference between Link State and Distance Vector Routing? Explain how these two algorithms help in identifying the least-cost path with the help of examples.

Answer:

Therefore, the least-cost path has a cost of **5**. The router compares the available paths and selects the path with the lowest cost.

ASSIGNMENT-II

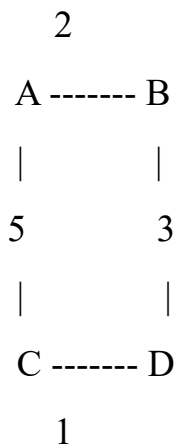
One commonly known Distance Vector routing protocol is **RIP (Routing Information Protocol)**.

Link State Routing

In **Link State Routing**, every router collects information about the links and their costs in the network. Using this information, the router creates a complete or sufficiently detailed topology of the network.

It then uses **Dijkstra's Shortest Path First (SPF) algorithm** to calculate the least-cost path.

Consider:



There are two possible paths from A to D:

Path 1:

$A \rightarrow B \rightarrow D$

Cost:

$$2 + 3 = 5$$

Path 2:

$A \rightarrow C \rightarrow D$

Cost:

$$5 + 1 = 6$$

Therefore, the least-cost path is:

$A \rightarrow B \rightarrow D$

with a total cost of **5**.

ASSIGNMENT-II

A common example of a Link State routing protocol is **OSPF (Open Shortest Path First)**.

Difference between Distance Vector and Link State Routing

Distance Vector

Router knows information mainly from its neighbors.

Uses Bellman-Ford algorithm.

Routing information is exchanged with neighbors.

Generally requires less memory and processing.

Convergence can be slower.

Example: RIP

Link State

Router builds a topology of the network.

Uses Dijkstra's algorithm.

Link-state information is distributed throughout the network.

Generally requires more memory and processing.

Generally provides faster convergence.

Example: OSPF

Thus, both algorithms help routers find **least-cost paths**, but they use different methods and types of network information.

Q3. What is congestion in a network? Why do we need to control congestion? Explain the difference between Leaky Bucket and Token Bucket with examples. Also, explain why do we need to do fragmentation?

Answer:

Network Congestion

Congestion occurs when the amount of data entering a network becomes greater than the network's ability to process and transmit that data.

For example, if a router can handle **100 Mbps** of traffic but receives **150 Mbps**, the extra traffic has to wait in buffers. If the buffers become full, packets may be dropped.

Causes of Congestion

The main causes of network congestion are:

- Large amount of network traffic.

ASSIGNMENT-II

- Limited bandwidth.
- Slow network devices.
- Limited buffer capacity.
- Sudden bursts of data.

Effects of Congestion

Congestion can result in:

- Increased network delay.
- Packet loss.
- Reduced throughput.
- Retransmission of packets.
- Poor network performance.

Therefore, **congestion control** is required to regulate the amount of traffic entering the network and to maintain efficient network performance.

Leaky Bucket Algorithm

The **Leaky Bucket algorithm** is a congestion control and traffic shaping technique in which packets are transmitted at a relatively fixed rate.

It can be compared to a bucket with a small hole at the bottom. Water may enter the bucket at different rates, but it leaves through the hole at a fixed rate.

Similarly, packets can arrive in bursts, but they are transmitted at a controlled rate.

Example:

Suppose a bucket can store 10 packets and allows 2 packets to leave every second. If 8 packets arrive at once, they are stored and transmitted at the rate of 2 packets per second. If the bucket becomes full and additional packets arrive, those packets may be discarded.

Token Bucket Algorithm

The **Token Bucket algorithm** controls traffic using tokens. Tokens are generated at a fixed rate and stored in a bucket.

ASSIGNMENT-II

A packet can be transmitted only when a sufficient number of tokens is available.

For example, suppose tokens are generated at a rate of **2 tokens per second** and the bucket can store up to 10 tokens. If 8 tokens have accumulated, the system can transmit a burst of 8 packets immediately, assuming one token is required for each packet.

Thus, the Token Bucket allows **controlled bursts of traffic** while maintaining a specified average transmission rate.

Difference between Leaky Bucket and Token Bucket

Leaky Bucket

Packets are stored in a queue/bucket. Tokens are stored in the bucket.

Generally produces a fixed output rate.

Smooths traffic more strictly.

Excess packets may be discarded when the bucket is full.

Suitable when a smooth transmission rate is required.

Token Bucket

Allows variable output and controlled bursts.

Allows bursty traffic.

Tokens are discarded when the bucket is full; packets can be sent when tokens are available.

Suitable when controlled bursts are acceptable.

Fragmentation

Fragmentation is the process of dividing a large IP packet into smaller packets called **fragments**.

Fragmentation is required because different networks may have different **MTU (Maximum Transmission Unit)** sizes.

For example, suppose a packet has a size of **4000 bytes**, but the next network can carry a maximum of **1500 bytes** in one packet. The original packet cannot be transmitted through that network as a single packet.

Therefore, it may be divided into smaller fragments.

4000-byte IP Packet

|

ASSIGNMENT-II

Fragmentation



Fragment	Fragment	Fragment
1	2	3

Network

Reassembly at
destination

The fragments contain information that allows the destination to identify and reassemble them into the original packet.

Need for Fragmentation

Fragmentation is required:

1. When a packet is larger than the MTU of the next network.
2. To allow packets to pass through networks with different MTU sizes.
3. To ensure that large packets can be transmitted over smaller-capacity links.

Important IPv4 fields associated with fragmentation include the **Identification field**, **Fragment Offset**, and **More Fragments (MF) flag**.